

Machine Learning Algorithms Laboratory

Experiment 7

Model Evaluation with and without PCA

Name: MAHALINGAM S S

Register No.: 3122247001034

Aim

To evaluate and compare multiple machine learning classification models with and without Principal Component Analysis (PCA) on the Wisconsin Diagnostic Breast Cancer (WDBC) dataset.

Objective

- Load and preprocess the WDBC dataset.
- Standardize the input features and determine the number of PCA components required to retain 95% variance.
- Train and tune SVM, Naive Bayes, KNN, Logistic Regression, Decision Tree, Random Forest, AdaBoost, Gradient Boosting, XGBoost and Stacking classifiers.
- Evaluate every model both without PCA and with PCA using 5-fold stratified cross-validation.
- Compare test-set accuracy, precision, recall and F1 score.
- Examine the best models using confusion matrices and a precision-recall curve.

Theory

Principal Component Analysis (PCA) is a dimensionality reduction technique that transforms the original features into a smaller set of uncorrelated principal components. The components are ordered according to the amount of variance they explain. In this experiment, PCA is used with a retained variance of 95%.

Standardization is performed before PCA so that features with different numerical scales contribute appropriately to the principal components. The StandardScaler is fitted on the training data inside the model pipeline.

Classification Models used in the experiment include SVM, Gaussian Naive Bayes, KNN, Logistic Regression, Decision Tree, Random Forest, AdaBoost, Gradient Boosting, XGBoost and a Stacking classifier. GridSearchCV is used to select suitable hyperparameters for each model.

Evaluation is carried out using accuracy, precision, recall and F1 score on the held-out test set. Five-fold StratifiedKFold cross-validation is used during model selection and comparison.

Dataset

The WDBC dataset is loaded from `wdbc.data`. It contains 569 samples, an ID column, a diagnosis column and 30 numerical input features. The diagnosis is converted to a binary target where M (Malignant) is mapped to 1 and B (Benign) is mapped to 0. The ID column is removed before model training.

The class distribution is 357 benign samples and 212 malignant samples. No missing values are present in the input features. The dataset is divided into 80% training data and 20% testing data using stratification.

Implementation

1. Installation and Imports

The XGBoost package is installed if required, followed by the libraries used for preprocessing, PCA, classification, hyperparameter tuning and evaluation.

```
1 %pip install xgboost
2
3 import pandas as pd
4 import numpy as np
5 import matplotlib.pyplot as plt
6
7 from sklearn.model_selection import (
8     train_test_split,
9     GridSearchCV,
10    StratifiedKFold,
11    cross_val_score
12 )
13
14 from sklearn.preprocessing import StandardScaler
15 from sklearn.decomposition import PCA
16 from sklearn.pipeline import Pipeline
17
18 from sklearn.svm import SVC
19 from sklearn.naive_bayes import GaussianNB
20 from sklearn.neighbors import KNeighborsClassifier
```

```

21 from sklearn.linear_model import LogisticRegression
22 from sklearn.tree import DecisionTreeClassifier
23 from sklearn.ensemble import (
24     RandomForestClassifier,
25     AdaBoostClassifier,
26     GradientBoostingClassifier,
27     StackingClassifier
28 )
29
30 from sklearn.metrics import (
31     accuracy_score,
32     precision_score,
33     recall_score,
34     f1_score,
35     confusion_matrix,
36     ConfusionMatrixDisplay,
37     classification_report,
38     roc_curve,
39     roc_auc_score,
40     precision_recall_curve,
41     average_precision_score
42 )
43
44 from xgboost import XGBClassifier
45
46 import warnings
47 warnings.filterwarnings("ignore")

```

2. Loading the Dataset

```

1  # Load WDBC dataset
2  data = pd.read_csv("wdbc.data", header=None)
3
4  # Create column names
5  columns = ["ID", "Diagnosis"] + [f"Feature_{i}" for i in range(1, 31)]
6  data.columns = columns
7
8  print("Dataset Shape:", data.shape)
9  print(data.head())

```

Output: Dataset Shape: (569, 32). The first five records contain the ID, diagnosis and 30 numerical features.

3. Target Conversion and Dataset Check

```

1  # Convert target:
2  # M = 1 (Malignant)
3  # B = 0 (Benign)

```

```

4
5 data["Diagnosis"] = data["Diagnosis"].map({
6     "M": 1,
7     "B": 0
8 })
9
10 # Remove ID
11 X = data.drop(columns=["ID", "Diagnosis"])
12 y = data["Diagnosis"]
13
14 print("Number of samples:", X.shape[0])
15 print("Number of features:", X.shape[1])
16
17 print("\nClass Distribution:")
18 print(y.value_counts())
19
20 print("\nMissing Values:", X.isnull().sum().sum())

```

Output:

- Number of samples: 569
- Number of features: 30
- Benign: 357
- Malignant: 212
- Missing values: 0

4. Train-Test Split

```

1 X_train, X_test, y_train, y_test = train_test_split(
2     X,
3     y,
4     test_size=0.20,
5     random_state=42,
6     stratify=y
7 )
8
9 print("Training samples:", X_train.shape[0])
10 print("Testing samples:", X_test.shape[0])

```

Output: Training samples: 455; Testing samples: 114.

5. PCA and Explained Variance

The training data is standardized before PCA. The cumulative explained variance is plotted to determine the number of components required to retain at least 95% of the variance.

```

1  # Standardize data before PCA
2  scaler = StandardScaler()
3
4  X_scaled = scaler.fit_transform(X_train)
5
6  # PCA with all components
7  pca_full = PCA()
8  pca_full.fit(X_scaled)
9
10 cumulative_variance = np.cumsum(
11     pca_full.explained_variance_ratio_
12 )
13
14 plt.figure(figsize=(8, 5))
15 plt.plot(
16     range(1, len(cumulative_variance) + 1),
17     cumulative_variance,
18     marker="o"
19 )
20
21 plt.axhline(
22     y=0.95,
23     linestyle="--",
24     label="95% Variance"
25 )
26
27 plt.xlabel("Number of Components")
28 plt.ylabel("Cumulative Explained Variance")
29 plt.title("PCA - Cumulative Explained Variance")
30 plt.legend()
31 plt.grid()
32 plt.show()

```

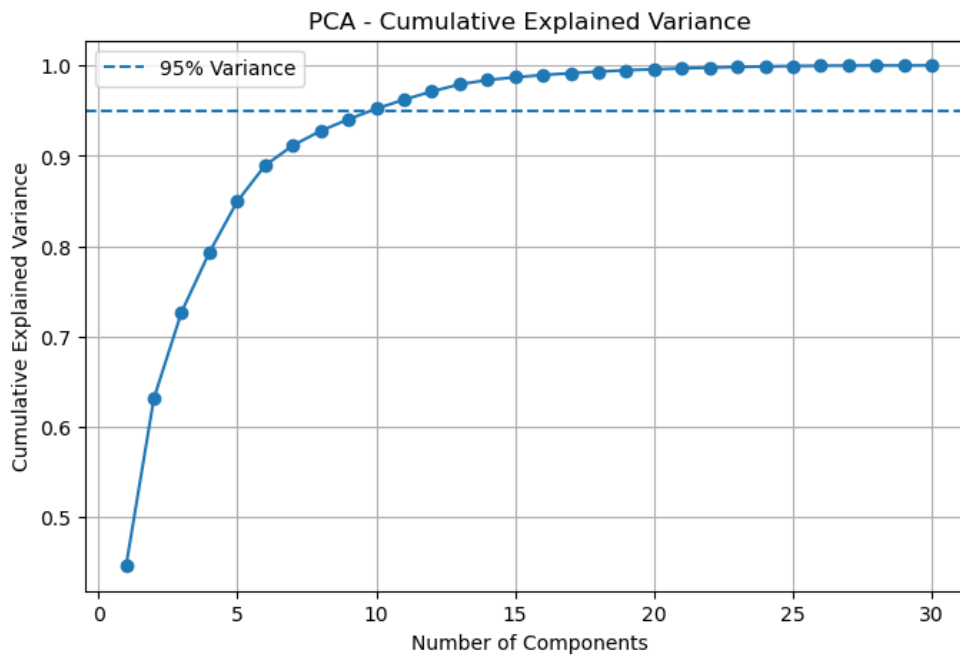


Figure 1: PCA cumulative explained variance

```

1  n_components = np.argmax(cumulative_variance >= 0.95) + 1
2
3  print("Components required for 95% variance:", n_components)
4
5  print(
6      "Explained variance:",
7      cumulative_variance[n_components - 1] * 100,
8      "%"
9  )

```

Output: 10 components are required to retain 95% variance. The explained variance with these components is 95.2069%.

6. Classification Models

```

1  models = {
2
3      "SVM": SVC(probability=True, random_state=42),
4
5      "Naive Bayes": GaussianNB(),
6
7      "KNN": KNeighborsClassifier(),
8
9      "Logistic Regression": LogisticRegression(
10         max_iter=2000,
11         random_state=42
12     ),
13

```

```

14     "Decision Tree": DecisionTreeClassifier(
15         random_state=42
16     ),
17
18     "Random Forest": RandomForestClassifier(
19         random_state=42
20     ),
21
22     "AdaBoost": AdaBoostClassifier(
23         random_state=42
24     ),
25
26     "Gradient Boosting": GradientBoostingClassifier(
27         random_state=42
28     ),
29
30     "XGBoost": XGBClassifier(
31         eval_metric="logloss",
32         random_state=42
33     )
34 }
35
36 stacking_model = StackingClassifier(
37     estimators=[
38         (
39             "svm",
40             SVC(
41                 probability=True,
42                 random_state=42
43             )
44         ),
45         (
46             "nb",
47             GaussianNB()
48         ),
49         (
50             "dt",
51             DecisionTreeClassifier(
52                 random_state=42
53             )
54         )
55     ],
56     final_estimator=LogisticRegression(
57         max_iter=2000
58     ),
59     cv=5
60 )
61
62 models["Stacking"] = stacking_model
63
64 cv = StratifiedKFold(
65     n_splits=5,

```

```

66     shuffle=True,
67     random_state=42
68 )

```

7. Pipeline for PCA and No-PCA Models

The following function creates a pipeline with standardization in both cases. PCA is included only for the With-PCA setting.

```

1  def create_pipeline(model, use_pca=False):
2
3      if use_pca:
4
5          pipeline = Pipeline([
6              ("scaler", StandardScaler()),
7              ("pca", PCA(n_components=0.95)),
8              ("model", model)
9          ])
10
11     else:
12
13         pipeline = Pipeline([
14             ("scaler", StandardScaler()),
15             ("model", model)
16         ])
17
18     return pipeline

```

8. Hyperparameter Grids

```

1  param_grids = {
2
3      "SVM": {
4          "model__C": [0.1, 1, 10],
5          "model__kernel": ["linear", "rbf"],
6          "model__gamma": ["scale", "auto"]
7      },
8
9      "Naive Bayes": {
10         "model__var_smoothing": [1e-9, 1e-8, 1e-7]
11     },
12
13     "KNN": {
14         "model__n_neighbors": [3, 5, 7, 9],
15         "model__weights": ["uniform", "distance"],
16         "model__metric": ["euclidean", "manhattan"]
17     },
18

```

```

19     "Logistic Regression": {
20         "model__C": [0.01, 0.1, 1, 10]
21     },
22
23     "Decision Tree": {
24         "model__max_depth": [None, 3, 5, 10],
25         "model__min_samples_split": [2, 5, 10]
26     },
27
28     "Random Forest": {
29         "model__n_estimators": [50, 100],
30         "model__max_depth": [None, 5, 10],
31         "model__max_features": ["sqrt", "log2"]
32     },
33
34     "AdaBoost": {
35         "model__n_estimators": [50, 100],
36         "model__learning_rate": [0.01, 0.1, 1]
37     },
38
39     "Gradient Boosting": {
40         "model__n_estimators": [50, 100],
41         "model__learning_rate": [0.05, 0.1],
42         "model__max_depth": [2, 3]
43     },
44
45     "XGBoost": {
46         "model__n_estimators": [50, 100],
47         "model__max_depth": [2, 3],
48         "model__learning_rate": [0.05, 0.1]
49     },
50
51     "Stacking": {
52         "model__final_estimator__C": [0.1, 1, 10]
53     }
54 }
55
56 print("Hyperparameter grids loaded successfully!")

```

9. Grid Search and Cross-Validation

Each classifier is evaluated in two settings: No-PCA and With-PCA. GridSearchCV is used with five-fold stratified cross-validation, followed by cross-validation using the selected best estimator.

```

1  best_models = {}
2  cv_results = []
3
4  for name, model in models.items():
5

```

```

6     print("\n" + "=" * 60)
7     print("MODEL:", name)
8     print("=" * 60)
9
10    for setting in ["No-PCA", "With-PCA"]:
11
12        print("\nRunning:", setting)
13
14        use_pca = setting == "With-PCA"
15
16        pipeline = create_pipeline(
17            model,
18            use_pca
19        )
20
21        grid = GridSearchCV(
22            estimator=pipeline,
23            param_grid=param_grids[name],
24            cv=cv,
25            scoring="accuracy",
26            n_jobs=-1
27        )
28
29        grid.fit(X_train, y_train)
30
31        best_models[(name, setting)] = grid.best_estimator_
32
33        # Get five CV fold scores using best model
34        fold_scores = cross_val_score(
35            grid.best_estimator_,
36            X_train,
37            y_train,
38            cv=cv,
39            scoring="accuracy",
40            n_jobs=-1
41        )
42
43        print("Best Parameters:", grid.best_params_)
44        print("Fold Scores:", fold_scores)
45        print("Average Accuracy:", fold_scores.mean())
46
47        cv_results.append({
48            "Model": name,
49            "Setting": setting,
50            "Fold 1": fold_scores[0],
51            "Fold 2": fold_scores[1],
52            "Fold 3": fold_scores[2],
53            "Fold 4": fold_scores[3],
54            "Fold 5": fold_scores[4],
55            "Average Accuracy": fold_scores.mean(),
56            "Std Dev": fold_scores.std()
57        })

```

10. Cross-Validation Results

```
1 cv_df = pd.DataFrame(cv_results)
2
3 print(
4     cv_df.to_string(index=False)
5 )
6
7 comparison = cv_df.pivot(
8     index="Model",
9     columns="Setting",
10    values="Average Accuracy"
11 )
12
13 print("\nAverage CV Accuracy Comparison:")
14 print(comparison.round(4))
15
16 comparison["Improvement (%)"] = (
17     comparison["With-PCA"]
18     -
19     comparison["No-PCA"]
20 ) * 100
21
22 print(
23     comparison.round(4)
24 )
```

Table 1: Average cross-validation accuracy comparison

Model	No-PCA	With-PCA	Improvement (%)
AdaBoost	0.9670	0.9714	0.4396
Decision Tree	0.9231	0.9341	1.0989
Gradient Boosting	0.9670	0.9516	-1.5385
KNN	0.9758	0.9714	-0.4396
Logistic Regression	0.9736	0.9758	0.2198
Naive Bayes	0.9385	0.9165	-2.1978
Random Forest	0.9670	0.9538	-1.3187
SVM	0.9758	0.9736	-0.2198
Stacking	0.9736	0.9714	-0.2198
XGBoost	0.9714	0.9648	-0.6593

The highest average cross-validation accuracy without PCA is obtained by SVM and KNN at 0.9758. With PCA, Logistic Regression gives the highest average cross-validation accuracy at 0.9758.

11. Test Set Evaluation

```
1 test_results = []
2
3 for name in models.keys():
4
5     for setting in ["No-PCA", "With-PCA"]:
6
7         model = best_models[(name, setting)]
8
9         y_pred = model.predict(X_test)
10
11        accuracy = accuracy_score(
12            y_test,
13            y_pred
14        )
15
16        precision = precision_score(
17            y_test,
18            y_pred
19        )
20
21        recall = recall_score(
22            y_test,
23            y_pred
24        )
25
26        f1 = f1_score(
27            y_test,
28            y_pred
29        )
30
31        test_results.append({
32            "Model": name,
33            "Setting": setting,
34            "Accuracy": accuracy,
35            "Precision": precision,
36            "Recall": recall,
37            "F1 Score": f1
38        })
39
40 test_df = pd.DataFrame(test_results)
41
42 print(
43     test_df.round(4).to_string(index=False)
44 )
45
46 accuracy_comparison = test_df.pivot(
47     index="Model",
48     columns="Setting",
49     values="Accuracy"
50 )
```

```

51
52 f1_comparison = test_df.pivot(
53     index="Model",
54     columns="Setting",
55     values="F1 Score"
56 )
57
58 print("ACCURACY")
59 print(accuracy_comparison.round(4))
60
61 print("\nF1 SCORE")
62 print(f1_comparison.round(4))

```

Table 2: Test-set performance for all models

Model	Setting	Accuracy(%)	Precision(%)	Recall(%)	F1(%)
SVM	No-PCA	97.37	100.00	92.86	96.30
SVM	With-PCA	98.25	100.00	95.24	97.56
Naive Bayes	No-PCA	92.11	92.31	85.71	88.89
Naive Bayes	With-PCA	89.47	85.71	85.71	85.71
KNN	No-PCA	96.49	100.00	90.48	95.00
KNN	With-PCA	94.74	97.37	88.10	92.50
Logistic Regression	No-PCA	96.49	97.50	92.86	95.12
Logistic Regression	With-PCA	97.37	100.00	92.86	96.30
Decision Tree	No-PCA	92.98	90.48	90.48	90.48
Decision Tree	With-PCA	93.86	94.87	88.10	91.36
Random Forest	No-PCA	97.37	100.00	92.86	96.30
Random Forest	With-PCA	93.86	92.68	90.48	91.57
AdaBoost	No-PCA	98.25	100.00	95.24	97.56
AdaBoost	With-PCA	96.49	97.50	92.86	95.12
Gradient Boosting	No-PCA	96.49	100.00	90.48	95.00
Gradient Boosting	With-PCA	95.61	97.44	90.48	93.83
XGBoost	No-PCA	96.49	100.00	90.48	95.00
XGBoost	With-PCA	96.49	97.50	92.86	95.12
Stacking	No-PCA	98.25	100.00	95.24	97.56
Stacking	With-PCA	97.37	100.00	92.86	96.30

12. Best Model Selection

```

1  # Find best model for No-PCA
2  best_no_pca = test_df[
3      test_df["Setting"] == "No-PCA"
4  ].sort_values(
5      "Accuracy",
6      ascending=False
7  ).iloc[0]
8
9  # Find best model for PCA
10 best_pca = test_df[
11     test_df["Setting"] == "With-PCA"
12 ].sort_values(
13     "Accuracy",
14     ascending=False
15 ).iloc[0]
16

```

```

17 print("Best No-PCA model:")
18 print(best_no_pca)
19
20 print("\nBest With-PCA model:")
21 print(best_pca)

```

The best No-PCA model on the test set is AdaBoost with 98.25% accuracy and 97.56% F1 score. The best With-PCA model is SVM with 98.25% accuracy and 97.56% F1 score.

13. Confusion Matrices

```

1  selected_models = [
2      (
3          best_no_pca["Model"],
4          "No-PCA"
5      ),
6      (
7          best_pca["Model"],
8          "With-PCA"
9      )
10 ]
11
12 for name, setting in selected_models:
13
14     model = best_models[(name, setting)]
15
16     y_pred = model.predict(X_test)
17
18     cm = confusion_matrix(
19         y_test,
20         y_pred
21     )
22
23     disp = ConfusionMatrixDisplay(
24         confusion_matrix=cm,
25         display_labels=["Benign", "Malignant"]
26     )
27
28     disp.plot()
29
30     plt.title(
31         f"{name} - {setting}"
32     )
33
34     plt.show()

```

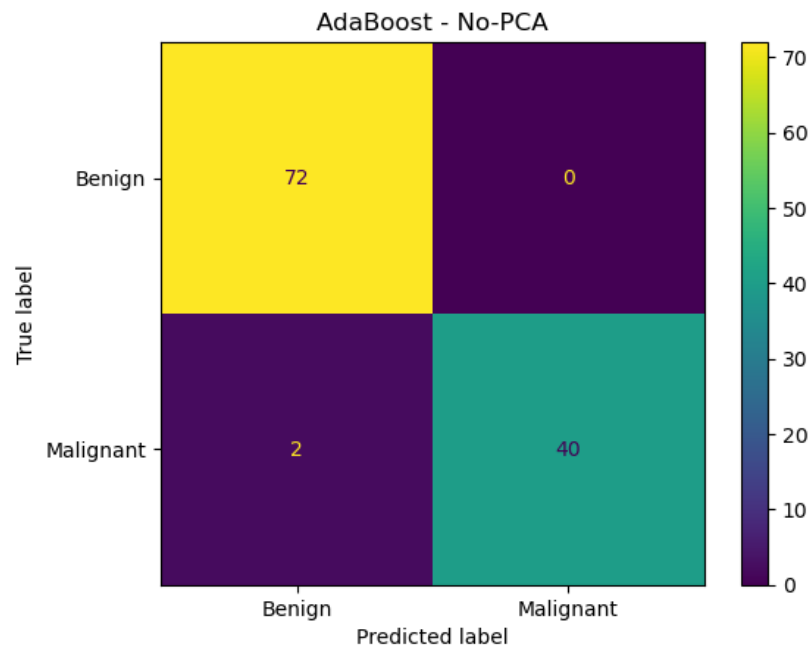


Figure 2: Confusion matrix for AdaBoost without PCA

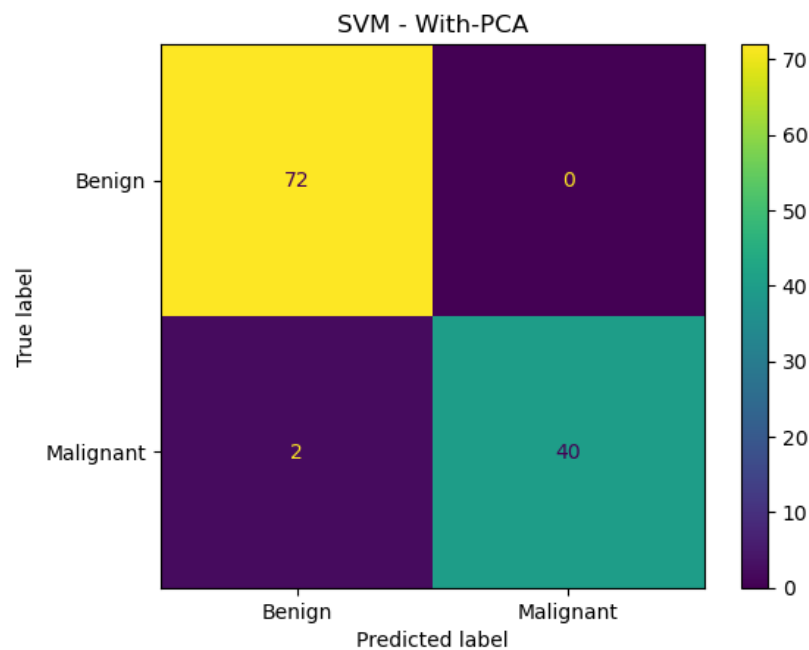


Figure 3: Confusion matrix for SVM with PCA

Both selected models classify 72 benign samples correctly and 40 malignant samples correctly, with 2 malignant samples classified as benign and no benign samples classified as malignant.

14. Precision-Recall Curve

```
plt.figure(figsize=(9, 6))
```

```

2
3 for name, setting in selected_models:
4
5     model = best_models[(name, setting)]
6
7     y_prob = model.predict_proba(X_test)[:, 1]
8
9     precision, recall, _ = precision_recall_curve(
10         y_test,
11         y_prob
12     )
13
14     ap = average_precision_score(
15         y_test,
16         y_prob
17     )
18
19     plt.plot(
20         recall,
21         precision,
22         label=f"{name} ({setting}) AP={ap:.3f}"
23     )
24
25 plt.xlabel("Recall")
26 plt.ylabel("Precision")
27
28 plt.title(
29     "Precision-Recall Curve"
30 )
31
32 plt.legend()
33 plt.grid()
34
35 plt.show()

```

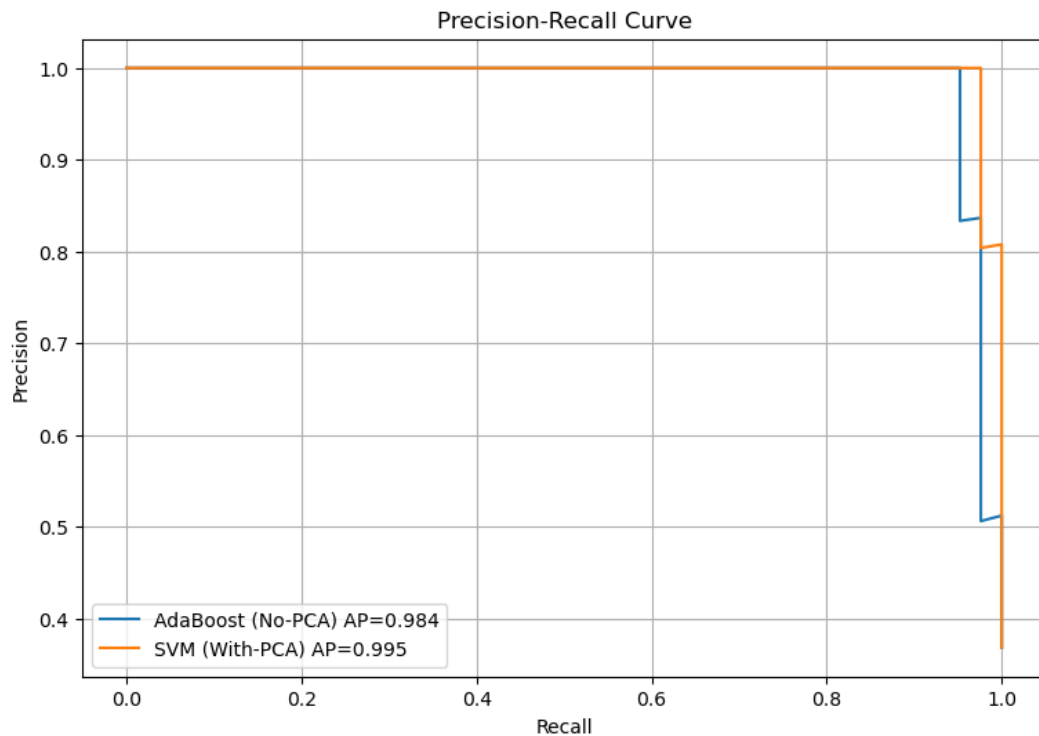


Figure 4: Precision-recall curve for the selected No-PCA and With-PCA models

The average precision values shown in the plot are 0.984 for AdaBoost without PCA and 0.995 for SVM with PCA.

15. Final Results Table

```

1 final_table = test_df.copy()
2
3 final_table["Accuracy (%)"] = (
4     final_table["Accuracy"] * 100
5 )
6
7 final_table["Precision (%)"] = (
8     final_table["Precision"] * 100
9 )
10
11 final_table["Recall (%)"] = (
12     final_table["Recall"] * 100
13 )
14
15 final_table["F1 Score (%)"] = (
16     final_table["F1 Score"] * 100
17 )
18
19 final_table = final_table[
20     [
21         "Model",

```

```
22         "Setting",
23         "Accuracy (%)",
24         "Precision (%)",
25         "Recall (%)",
26         "F1 Score (%)"
27     ]
28 ]
29
30 print(
31     final_table.round(2).to_string(index=False)
32 )
```

Conclusion

The experiment compared ten classification approaches with and without PCA on the WDBC dataset. PCA reduced the original 30 features to 10 components while retaining 95.2069% of the variance. Its effect was model-dependent: it improved the test accuracy of SVM, Logistic Regression and Decision Tree, while reducing the performance of several tree-based and distance-based models.

On the test set, AdaBoost without PCA, SVM with PCA and Stacking without PCA achieved the highest accuracy of 98.25%. Among the two selected best models, SVM with PCA produced an average precision of 0.995, compared with 0.984 for AdaBoost without PCA. Thus, PCA can reduce dimensionality substantially while maintaining very strong classification performance, but the benefit depends on the classifier used.

Complete Code

The complete notebook code used for the experiment is reproduced below in the same order as the implementation.

Code Cell 0

```
1 %pip install xgboost
```

Code Cell 1

```
1 import pandas as pd
2 import numpy as np
3 import matplotlib.pyplot as plt
4
5 from sklearn.model_selection import (
6     train_test_split,
7     GridSearchCV,
8     StratifiedKFold,
9     cross_val_score
10 )
11
12 from sklearn.preprocessing import StandardScaler
13 from sklearn.decomposition import PCA
14 from sklearn.pipeline import Pipeline
15
16 from sklearn.svm import SVC
17 from sklearn.naive_bayes import GaussianNB
18 from sklearn.neighbors import KNeighborsClassifier
19 from sklearn.linear_model import LogisticRegression
20 from sklearn.tree import DecisionTreeClassifier
21 from sklearn.ensemble import (
22     RandomForestClassifier,
23     AdaBoostClassifier,
24     GradientBoostingClassifier,
25     StackingClassifier
26 )
27
28 from sklearn.metrics import (
29     accuracy_score,
30     precision_score,
31     recall_score,
32     f1_score,
33     confusion_matrix,
34     ConfusionMatrixDisplay,
35     classification_report,
36     roc_curve,
37     roc_auc_score,
```

```

38     precision_recall_curve,
39     average_precision_score
40 )
41
42 from xgboost import XGBClassifier
43
44 import warnings
45 warnings.filterwarnings("ignore")

```

Code Cell 2

```

1  # Load WDBC dataset
2  data = pd.read_csv("wdbc.data", header=None)
3
4  # Create column names
5  columns = ["ID", "Diagnosis"] + [f"Feature_{i}" for i in range(1, 31)]
6  data.columns = columns
7
8  print("Dataset Shape:", data.shape)
9  print(data.head())

```

Code Cell 3

```

1  # Convert target:
2  # M = 1 (Malignant)
3  # B = 0 (Benign)
4
5  data["Diagnosis"] = data["Diagnosis"].map({
6      "M": 1,
7      "B": 0
8  })
9
10 # Remove ID
11 X = data.drop(columns=["ID", "Diagnosis"])
12 y = data["Diagnosis"]
13
14 print("Number of samples:", X.shape[0])
15 print("Number of features:", X.shape[1])
16
17 print("\nClass Distribution:")
18 print(y.value_counts())
19
20 print("\nMissing Values:", X.isnull().sum().sum())

```

Code Cell 4

```

1 X_train, X_test, y_train, y_test = train_test_split(
2     X,
3     y,
4     test_size=0.20,
5     random_state=42,
6     stratify=y
7 )
8
9 print("Training samples:", X_train.shape[0])
10 print("Testing samples:", X_test.shape[0])

```

Code Cell 5

```

1 # Standardize data before PCA
2 scaler = StandardScaler()
3
4 X_scaled = scaler.fit_transform(X_train)
5
6 # PCA with all components
7 pca_full = PCA()
8 pca_full.fit(X_scaled)
9
10 cumulative_variance = np.cumsum(
11     pca_full.explained_variance_ratio_
12 )
13
14 plt.figure(figsize=(8, 5))
15 plt.plot(
16     range(1, len(cumulative_variance) + 1),
17     cumulative_variance,
18     marker="o"
19 )
20
21 plt.axhline(
22     y=0.95,
23     linestyle="--",
24     label="95% Variance"
25 )
26
27 plt.xlabel("Number of Components")
28 plt.ylabel("Cumulative Explained Variance")
29 plt.title("PCA - Cumulative Explained Variance")
30 plt.legend()
31 plt.grid()
32 plt.show()

```

Code Cell 6

```

1  n_components = np.argmax(cumulative_variance >= 0.95) + 1
2
3  print("Components required for 95% variance:", n_components)
4
5  print(
6      "Explained variance:",
7      cumulative_variance[n_components - 1] * 100,
8      "%"
9  )

```

Code Cell 7

```

1  models = {
2
3      "SVM": SVC(probability=True, random_state=42),
4
5      "Naive Bayes": GaussianNB(),
6
7      "KNN": KNeighborsClassifier(),
8
9      "Logistic Regression": LogisticRegression(
10         max_iter=2000,
11         random_state=42
12     ),
13
14     "Decision Tree": DecisionTreeClassifier(
15         random_state=42
16     ),
17
18     "Random Forest": RandomForestClassifier(
19         random_state=42
20     ),
21
22     "AdaBoost": AdaBoostClassifier(
23         random_state=42
24     ),
25
26     "Gradient Boosting": GradientBoostingClassifier(
27         random_state=42
28     ),
29
30     "XGBoost": XGBClassifier(
31         eval_metric="logloss",
32         random_state=42
33     )
34 }

```

Code Cell 8

```
1  stacking_model = StackingClassifier(  
2      estimators=[  
3          (  
4              "svm",  
5              SVC(  
6                  probability=True,  
7                  random_state=42  
8              )  
9          ),  
10         (  
11             "nb",  
12             GaussianNB()  
13         ),  
14         (  
15             "dt",  
16             DecisionTreeClassifier(  
17                 random_state=42  
18             )  
19         )  
20     ],  
21     final_estimator=LogisticRegression(  
22         max_iter=2000  
23     ),  
24     cv=5  
25 )  
26  
27 models["Stacking"] = stacking_model
```

Code Cell 9

```
1  cv = StratifiedKFold(  
2      n_splits=5,  
3      shuffle=True,  
4      random_state=42  
5  )
```

Code Cell 10

```
1  def create_pipeline(model, use_pca=False):  
2  
3      if use_pca:  
4  
5          pipeline = Pipeline([  
6              ("scaler", StandardScaler()),  
7              ("pca", PCA(n_components=0.95)),  
8              ("model", model)
```

```

9         ])
10
11     else:
12
13         pipeline = Pipeline([
14             ("scaler", StandardScaler()),
15             ("model", model)
16         ])
17
18     return pipeline

```

Code Cell 11

```

1  param_grids = {
2
3      "SVM": {
4          "model__C": [0.1, 1, 10],
5          "model__kernel": ["linear", "rbf"],
6          "model__gamma": ["scale", "auto"]
7      },
8
9      "Naive Bayes": {
10         "model__var_smoothing": [1e-9, 1e-8, 1e-7]
11     },
12
13     "KNN": {
14         "model__n_neighbors": [3, 5, 7, 9],
15         "model__weights": ["uniform", "distance"],
16         "model__metric": ["euclidean", "manhattan"]
17     },
18
19     "Logistic Regression": {
20         "model__C": [0.01, 0.1, 1, 10]
21     },
22
23     "Decision Tree": {
24         "model__max_depth": [None, 3, 5, 10],
25         "model__min_samples_split": [2, 5, 10]
26     },
27
28     "Random Forest": {
29         "model__n_estimators": [50, 100],
30         "model__max_depth": [None, 5, 10],
31         "model__max_features": ["sqrt", "log2"]
32     },
33
34     "AdaBoost": {
35         "model__n_estimators": [50, 100],
36         "model__learning_rate": [0.01, 0.1, 1]
37     },

```

```

38
39     "Gradient Boosting": {
40         "model__n_estimators": [50, 100],
41         "model__learning_rate": [0.05, 0.1],
42         "model__max_depth": [2, 3]
43     },
44
45     "XGBoost": {
46         "model__n_estimators": [50, 100],
47         "model__max_depth": [2, 3],
48         "model__learning_rate": [0.05, 0.1]
49     },
50
51     "Stacking": {
52         "model__final_estimator__C": [0.1, 1, 10]
53     }
54 }
55
56 print("Hyperparameter grids loaded successfully!")

```

Code Cell 12

```

1  best_models = {}
2  cv_results = []
3
4  for name, model in models.items():
5
6      print("\n" + "=" * 60)
7      print("MODEL:", name)
8      print("=" * 60)
9
10     for setting in ["No-PCA", "With-PCA"]:
11
12         print("\nRunning:", setting)
13
14         use_pca = setting == "With-PCA"
15
16         pipeline = create_pipeline(
17             model,
18             use_pca
19         )
20
21         grid = GridSearchCV(
22             estimator=pipeline,
23             param_grid=param_grids[name],
24             cv=cv,
25             scoring="accuracy",
26             n_jobs=-1
27         )
28

```

```

29     grid.fit(X_train, y_train)
30
31     best_models[(name, setting)] = grid.best_estimator_
32
33     # Get five CV fold scores using best model
34     fold_scores = cross_val_score(
35         grid.best_estimator_,
36         X_train,
37         y_train,
38         cv=cv,
39         scoring="accuracy",
40         n_jobs=-1
41     )
42
43     print("Best Parameters:", grid.best_params_)
44     print("Fold Scores:", fold_scores)
45     print("Average Accuracy:", fold_scores.mean())
46
47     cv_results.append({
48         "Model": name,
49         "Setting": setting,
50         "Fold 1": fold_scores[0],
51         "Fold 2": fold_scores[1],
52         "Fold 3": fold_scores[2],
53         "Fold 4": fold_scores[3],
54         "Fold 5": fold_scores[4],
55         "Average Accuracy": fold_scores.mean(),
56         "Std Dev": fold_scores.std()
57     })

```

Code Cell 13

```

1  cv_df = pd.DataFrame(cv_results)
2
3  print(
4      cv_df.to_string(index=False)
5  )

```

Code Cell 14

```

1  comparison = cv_df.pivot(
2      index="Model",
3      columns="Setting",
4      values="Average Accuracy"
5  )
6
7  print("\nAverage CV Accuracy Comparison:")
8  print(comparison.round(4))

```

Code Cell 15

```
1 comparison["Improvement (%)"] = (  
2     comparison["With-PCA"]  
3     -  
4     comparison["No-PCA"]  
5 ) * 100  
6  
7 print(  
8     comparison.round(4)  
9 )
```

Code Cell 16

```
1 test_results = []  
2  
3 for name in models.keys():  
4  
5     for setting in ["No-PCA", "With-PCA"]:  
6  
7         model = best_models[(name, setting)]  
8  
9         y_pred = model.predict(X_test)  
10  
11         accuracy = accuracy_score(  
12             y_test,  
13             y_pred  
14         )  
15  
16         precision = precision_score(  
17             y_test,  
18             y_pred  
19         )  
20  
21         recall = recall_score(  
22             y_test,  
23             y_pred  
24         )  
25  
26         f1 = f1_score(  
27             y_test,  
28             y_pred  
29         )  
30  
31         test_results.append({  
32             "Model": name,  
33             "Setting": setting,  
34             "Accuracy": accuracy,  
35             "Precision": precision,  
36             "Recall": recall,
```

```

37         "F1 Score": f1
38     })
39
40 test_df = pd.DataFrame(test_results)
41
42 print(
43     test_df.round(4).to_string(index=False)
44 )

```

Code Cell 17

```

1 accuracy_comparison = test_df.pivot(
2     index="Model",
3     columns="Setting",
4     values="Accuracy"
5 )
6
7 f1_comparison = test_df.pivot(
8     index="Model",
9     columns="Setting",
10    values="F1 Score"
11 )
12
13 print("ACCURACY")
14 print(accuracy_comparison.round(4))
15
16 print("\nF1 SCORE")
17 print(f1_comparison.round(4))

```

Code Cell 18

```

1 # Find best model for No-PCA
2 best_no_pca = test_df[
3     test_df["Setting"] == "No-PCA"
4 ].sort_values(
5     "Accuracy",
6     ascending=False
7 ).iloc[0]
8
9 # Find best model for PCA
10 best_pca = test_df[
11     test_df["Setting"] == "With-PCA"
12 ].sort_values(
13     "Accuracy",
14     ascending=False
15 ).iloc[0]
16
17 print("Best No-PCA model:")

```

```
18 print(best_no_pca)
19
20 print("\nBest With-PCA model:")
21 print(best_pca)
```

Code Cell 19

```
1  selected_models = [
2      (
3          best_no_pca["Model"],
4          "No-PCA"
5      ),
6      (
7          best_pca["Model"],
8          "With-PCA"
9      )
10 ]
11
12 for name, setting in selected_models:
13
14     model = best_models[(name, setting)]
15
16     y_pred = model.predict(X_test)
17
18     cm = confusion_matrix(
19         y_test,
20         y_pred
21     )
22
23     disp = ConfusionMatrixDisplay(
24         confusion_matrix=cm,
25         display_labels=["Benign", "Malignant"]
26     )
27
28     disp.plot()
29
30     plt.title(
31         f"{name} - {setting}"
32     )
33
34     plt.show()
```

Code Cell 20

```
1  plt.figure(figsize=(9, 6))
2
3  for name, setting in selected_models:
4
```

```

5     model = best_models[(name, setting)]
6
7     y_prob = model.predict_proba(X_test)[: , 1]
8
9     precision, recall, _ = precision_recall_curve(
10         y_test,
11         y_prob
12     )
13
14     ap = average_precision_score(
15         y_test,
16         y_prob
17     )
18
19     plt.plot(
20         recall,
21         precision,
22         label=f"{name} ({setting}) AP={ap:.3f}"
23     )
24
25     plt.xlabel("Recall")
26     plt.ylabel("Precision")
27
28     plt.title(
29         "Precision-Recall Curve"
30     )
31
32     plt.legend()
33     plt.grid()
34
35     plt.show()

```

Code Cell 21

```

1  final_table = test_df.copy()
2
3  final_table["Accuracy (%)"] = (
4      final_table["Accuracy"] * 100
5  )
6
7  final_table["Precision (%)"] = (
8      final_table["Precision"] * 100
9  )
10
11 final_table["Recall (%)"] = (
12     final_table["Recall"] * 100
13 )
14
15 final_table["F1 Score (%)"] = (
16     final_table["F1 Score"] * 100

```

```
17 )
18
19 final_table = final_table[
20     [
21         "Model",
22         "Setting",
23         "Accuracy (%)",
24         "Precision (%)",
25         "Recall (%)",
26         "F1 Score (%)"
27     ]
28 ]
29
30 print(
31     final_table.round(2).to_string(index=False)
32 )
```